

Anomaly Detection in Microservices Architecture Using Graph Neural Networks

Matthias Osswald*, Timothy Schönenberger*, Gokcan Cantali*[‡], Wissem Soussi*[‡], and Gürkan Gür*

* Zurich University of Applied Sciences (ZHAW) InIT, CH-8401 Winterthur, Switzerland

[‡] Communication Systems Group CSG, University of Zürich UZH, CH-8050 Zürich, Switzerland
{name.surname}@zhaw.ch, matthias@, timothy@

Abstract—The rise of cloud computing has transformed application development and deployment, with Kubernetes emerging as a key platform for managing containerized applications. This paper explores the use of Graph Neural Networks (GNNs) to detect anomalies in Kubernetes clusters and proposes GNN-based Anomaly Detection in Kubernetes (GATAKU), which is instrumental in maintaining security and performance. Our work involves integrating Cilium for detailed network monitoring and data collection, setting up a Kubernetes cluster with k3s and Traefik, and simulating attack scenarios to generate realistic data. Data preprocessing and feature engineering prepare this data for the GNN training. We present the GATAKU model's performance, highlighting metrics such as accuracy, precision, recall, and F1-score and compare it to baseline ML models, namely Support Vector Machine (SVM) and Random Forest (RF). Moreover, we discuss these findings and emerging challenges, including handling high-dimensional data, and explore practical implications for cybersecurity.

I. INTRODUCTION

The rise of cloud computing has revolutionized the way applications are developed, deployed, and managed. The cloud-native technologies are also envisaged to be a fundamental part of future networks such as 6G/NextG systems and global digital infrastructure. Central to this revolution is the concept of containerization, with Kubernetes emerging as the leading orchestration platform for managing containerized applications [1]. Kubernetes clusters facilitate the deployment of microservices, enabling scalable and resilient application architectures. However, the dynamic and complex nature of these clusters introduces significant challenges in monitoring and maintaining their security and performance. A major challenge in this microservices environment is the efficient and accurate detection of anomalies indicating security and/or performance issues.

The overarching goal of this work is to forge a machine learning (ML) based model using Graph Neural Networks (GNNs) [2] that can efficiently and accurately identify potential threats and anomalies within a Kubernetes cluster. Anomalies can indicate potential security threats or performance issues, making their timely detection crucial for maintaining the reliability and security of cloud-native applications. To this end, we design and implement a dynamic graph model that captures the essence of the Kubernetes cluster's operations. This model is expected to evolve, mirroring changes within the cluster to reflect its state. To validate the effectiveness of the GNN-based Anomaly Detection in Kubernetes (GATAKU)

framework in a controlled environment, we simulate three primary types of attacks, namely unauthorized port scanning for reconnaissance attacks, Denial of Service (DoS) attacks, and web application scanning. We implement the GNN model to detect anomalies by analyzing the generated graph for unexpected changes, such as the emergence of new nodes or edges in the graph, or significant alterations in traffic patterns among nodes, which could indicate malicious activity or system faults. Through comprehensive testing and validation, we evaluate the model's predictive accuracy. We also measure its ability to operate under varying network load and attack complexity conditions and compare it to other techniques.

The motivation for this study stems from the need for more research regarding anomaly detection techniques targeting inter-container communication in a cloud environment. Moreover, there are few public datasets consisting of anomalous container network data. To alleviate these issues, our aim is to deliver a performant anomaly detection mechanism focusing on a cloud-native platform, namely Kubernetes, while leveraging modern ML methods to achieve our goal.

Our contributions in this work can be summarized as follows:

- Development and open-source implementation of an anomaly detection scheme using GNNs, namely GATAKU, and a synthetically generated dataset for containerized environments [3]
- An application of explainability techniques for the developed GNN-based detection model
- Performance evaluation through simulated experiments and comparison against traditional ML models

Section II provides an overview of related work regarding container security, GNN, and anomaly detection. Section III discusses the methodology used in GATAKU regarding design and implementation, describing infrastructure, technologies, and data collection process. Section IV presents the experimental results. Section V discusses our main findings and analyzes the GATAKU model decisions from the explainability perspective. Finally, Section VI concludes the paper.

¹GATAKU source code and dataset are available at the anonymous URL: <https://www.dropbox.com/scl/fi/sxk6cfkx38bqptkjtfg/GATAKU.zip?rlkey=cly256sznmw9iiqccwyjennz0&st=elserg6e&dl=0>

II. RELATED WORK

There exists a substantial body of research for anomaly detection techniques in the cloud, leveraging machine learning models such as random forest [3] and SVM [4]. With the advancements of deep learning, researchers have shifted their attention to neural networks for cloud-based anomaly detection as well. In [5], S. Naseer et al. explored various Deep Neural Network (DNN) architectures to enhance anomaly detection techniques. Siegel merged traditional ML techniques with advanced neural network architectures as a potential technique for anomaly detection [6]. Even though these studies showcase the superior performance of deep learning techniques compared to traditional ML models in detecting anomalous network traffic, there is a lack of usage for the underlying graph structure of the network data.

Hwang et al. [7] provided an in-depth survey of the current state and challenges in graph anomaly detection using GNNs. The study delved into various GNN architectures like Graph Convolutional Networks (GCN) and Graph Attention Networks (GAT). In one of the studies mentioned in the survey, Qiu et al. [8] combined deep one-class classification with graph transformation learning to propose one-class graph transformation learning, targeting performance flip problems that are frequently encountered in GNNs. In [9], a new deep learning model called Hybrid-Order Graph Attention Network was proposed for handling multiple anomaly detection tasks. Graph Attention-based Anomaly Detection (GATAE) by You et al. [10] focused on dealing with the over-smoothing problem. Pei et al. [11] proposed a novel approach for anomaly detection on attributed networks that tackles many challenges, including sparsity and nonlinearity capturing. In [12], a hop-count-based model was implemented, utilizing both global and local contextual information of nodes rather than relying on a single aspect. In [13], Kisanga et al. introduced the Activity and Event Network model, a novel graph-based framework for network anomaly detection that extends beyond traditional security tools. Although these works achieved remarkable results by utilizing GNNs for anomaly detection, they are not tailored for containers.

Although GNN-based anomaly detection techniques for container traffic are not a very active research area, other methods are extensively used for containerized environments. Tien et al. [14] developed KubAnomaly, a security system that aims to improve the security of Docker in the Kubernetes environment, using the frequency of events in a container. Another study was conducted in [15], where a dynamic anomaly detection scheme was developed for finding vulnerability exploits in containers. Container-level anomaly-based IDS is also a popular topic that was studied in [16] and [17]. Unlike many other studies, Molan et al. both employed GNNs and considered containers in [18]. The study proposed a novel approach to detect anomalies and displayed significant results. Nevertheless, the focus was on the anomaly of a high-performing computation process rather than a security perspective and, thus, lacked any concrete experiments on

cybersecurity datasets or attack scenarios.

III. GATAKU: DESIGN, METHODOLOGY AND IMPLEMENTATION

We employ a two-phased methodology to develop GATAKU. First, we build a Kubernetes cluster on our cloud environment and implement manually triggered attack scenarios to collect data for training ML models. Then, using the obtained data, we develop a supervised GCN model to detect anomalies in the monitored network data. Our model is later evaluated with comprehensive experiments.

A. Infrastructure Setup for GATAKU

For the data collection phase, we set up a Kubernetes cluster in an OpenStack-based cloud environment. The cluster is responsible for hosting a simple web application that can be publicly accessed.

1) *Testbed*: Our Kubernetes setup consists of five virtual machines (VMs), one of which acts as the control plane, and the rest take the role of nodes. Note that in this setup, the control-plane is not restricted to only managing the nodes, as it also undertakes some workload, allowing us to utilize all VMs in our cluster for running pods. The resource specification of our setup is shown in Table I.

Table I: Cloud Infrastructure Specifications

VM Role	OS	CPU	RAM	Storage
Control-plane VM	Ubuntu 22.04	8 vCPU	16 GB	160 GB
Node VMs (x4)	Ubuntu 22.04	4 vCPU	8 GB	80 GB

2) *Implementation Details*: We choose k3s for deploying and managing our distributed Kubernetes cluster, due to its optimized resource consumption in limited-scale production systems, especially for edge computing scenarios. As the CNI plugin of the Kubernetes cluster, we choose to leverage Cilium [19], which is built on top of extended Berkeley Packet Filter (eBPF) that allows user-defined safe programs to be executed in the kernel space [20].

Along with the base Cilium installation, we also incorporate a Hubble collector, aiming to extend the functionality of Cilium and provide deep network observability. Hubble makes it possible to monitor and gather network flows in our cluster for various layers, such as L3 and L4 of the TCP/IP stack [21]. To standardize the Cilium data, Hubble-collector is used, transforming the Cilium traces into an OpenTelemetry-compliant format. Furthermore, a custom OpenTelemetry backend is implemented in such a way that the Hubble-collector instances send the obtained data, particularly network traces provided by Cilium and Hubble, to this backend for data storage.

A high-level view of our cloud infrastructure, along with the utilized technology stack, can be observed in Figure 1 where uni-directed gray lines represent the data flow from the network to the custom OpenTelemetry backend via Hubble collectors, and undirected blue lines represent the internal network connections established by Kubernetes. Moreover, the telescope icon indicates that the transferred data is compliant with the OpenTelemetry format.

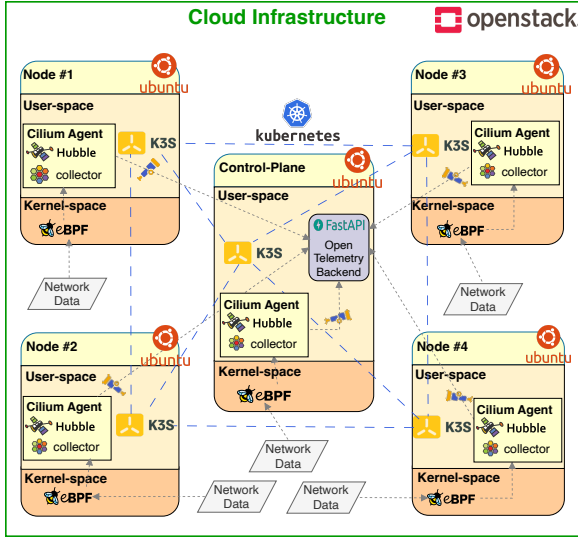


Figure 1: A high-level view of our cloud setup

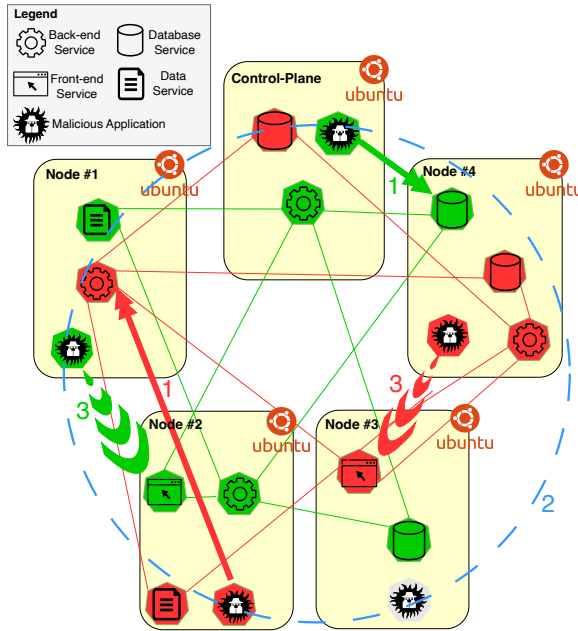


Figure 2: A sample deployment of our application landscape with two namespaces and visualized attack scenarios where 1, 2, and 3 correspond to DoS attacks, Port Scan attacks, and ZAP Scan attacks, respectively.

3) *Application Landscape*: In our Kubernetes cluster, we deploy a simple web-based application comprising the following components:

- A front-end service, running two instances that are exposed to the public internet via a Traefik Ingress.
- A back-end service, running four instances receiving requests from the front-end service while sending requests to a data service and a database service.
- A database (i.e. postgres) service, with three instances,

receives requests from the back-end service.

- A data service, with two instances, receives requests from the back-end service while sending requests to an external service outside the cluster.

Each service utilizes curl commands to periodically send requests to the services they are supposed to interact with. In addition, we employ the concept of namespaces in our cluster, to simulate a multi-tenant environment where different instances of the same application are isolated from each other. For instance, pods belonging to the front-end service in one namespace only establish communication with the pods belonging to the back-end service in the same namespace.

While our application consists of simple components and straightforward logic compared to more complex real-world scenarios, such architecture allows us to simulate in-cluster network flows among various pods and, by extension, different nodes. Note that the assignment of pods to nodes in the cluster is handled by the control-plane automatically and out of our control. Therefore, using a multi-namespace deployment plus pod replicas that are likely distributed amongst multiple nodes for each service allows us to increase the complexity of network flows, and increases the chance for each node to establish communication with every other node.

A sample instance of our application landscape is visualized in Figure 2. In this simplified illustration, the color of each pod indicates which namespace that the pod belongs to, and a thin arrow of a specific color represents the communication among pods within the same namespace. As one can observe, red (or green) pods only communicate with other red (or green) pods, enabling a multi-tenant environment where each deployment of the same application is isolated from each other.

4) *Attack Scenarios*: For simulating anomalous network traffic, we manually trigger some attack flows regularly during the data collection phase by spinning up malicious pods into the cluster. To distinguish malicious traffic from normal traffic, we mark each packet with a custom label representing whether the request coming from each pod is malicious or not, and another label representing the type of attack, if applicable.

In our setup, we develop and implement three different attack scenarios: DoS, Port Scan, and ZAP Scan, which are labeled in Figure 2 as 1, 2, and 3, respectively.

1. *DoS*: In DoS attack scenarios, we spin up a malicious pod for each namespace in the system. Then, each DoS attack performing pod randomly chooses a target benign pod in the same namespace and starts overwhelming the benign pod with incoming requests. For instance, the malicious pod in the red namespace might target a back-end pod in the red namespace, while the malicious pod in the green namespace might target a database pod in the green namespace, as illustrated in Figure 2.

For the DoS attack simulation, attack pods utilize the hey application, which is used to send concurrent HTTP requests to legitimate pods. Each attack wave lasts for about 30 seconds, during which each malicious pod continuously sends HTTP requests in 50 parallel threads.

2. *Port Scan*: Similar to the DoS scenario, we also spin up a malicious pod, but in only one random namespace, while

initiating a Port Scan attack. The pod then issues an `nmap` command on the subnet that the pod resides in and conducts a TCP SYN scan across the subnet. This scanning is quite common among cyberattackers, in an attempt to learn about the open ports and services in the network.

A wave of port scanning attack lasts until all 65535 ports of all possible IP addresses in the subnet are scanned through. Thus, even a single wave of this attack results in the collection of a significant amount of malicious data.

3. *ZAP Scan*: As the third attack scenario, we employ the OWASP ZAP tool, which is used to scan vulnerabilities of an application. During the attack, a malicious pod is spun up in every namespace and proceeds to scan a target benign pod. However, unlike a regular port scanning case, the ZAP Scan process focuses on scanning the vulnerabilities of a web application. Therefore, the target pods of this attack type specifically belong to front-end services.

One wave of ZAP scan attack lasts until the ZAP Scan process produces a report on the vulnerability of the target web application. Unfortunately, due to the limited scope of this type of scan, not as much data as in other attack scenarios is generated during a ZAP Scan attack.

Although there is a vast range of possible attack scenarios in a Kubernetes environment, many sophisticated threats come from specific vulnerabilities of certain applications or services. In this study, to prevent loss of generality, we focus on the early phases of the cyber-kill chain [22], which contains reconnaissance types of threats, hence the adoption of port scan and ZAP scan attacks. In addition, we also choose to include DoS attacks in our experiments, since they are generally applicable to most systems.

B. GNN Model Development

In the second phase of our methodology, we develop a suitable GNN model for detecting anomalous data in a containerized cloud environment. The development of such a model requires the execution of several tasks.

1) *Data Preprocessing*: The first step through a successful ML model is to develop a data preprocessing pipeline. Our input data should be compliant with the assumptions made by our GNN training library, in this case, PyTorch.

a) *Feature Encoders*: Due to the way tensors work in PyTorch, each feature available in our dataset needs to be represented as a numerical or categorical value. Therefore, we implement the following encoders and normalizers:

- IP encoder: For *ip_source* and *ip_destination*
- String encoder: For any string-values
- Number normalizer: *port_source* and *port_destination*
- Boolean encoder (i.e. String-to-Integer converter): For *ack_flag* and *psh_flag*

While the rest of the encoders and converters are common and straightforward, encoding IP values proves to be non-trivial. The IP encoder normalizes the numeric representation of IP addresses. For IPv6 addresses, each part of the address was converted to an integer and normalized by the maximum

possible value (65535). Similarly, IPv4 addresses were normalized by 255. The encoded IP addresses were then expanded into separate columns representing each part of the address, facilitating their use in the model.

b) *Feature Engineering*: In addition to the existing features available in the collected data, we enrich our dataset by introducing a new feature named *diversity_index*, which is calculated as follows:

- 1) The *timestamp* feature is converted into a DateTime format with UTC time zone, allowing us to make a consistent temporal comparison.
- 2) The dataset is sorted based on *ip_source* and *timestamp*.
- 3) A sliding time window of $t_w = 10$ seconds is applied to analyze the variety of ports accessed by each source IP within this interval.
- 4) For each source IP, the *diversity_index* is calculated as the count of unique destination ports accessed within each 10-second window.
- 5) After computing the *diversity_index* for each time window, the values are aggregated and merged back into the main dataset. A zero padding approach is employed for empty values.

This new *diversity_index* feature provides insight into malicious flows that require consecutive packet transmission from a single source.

2) *Graph Construction*: Unlike most problems GNNs tackle, such as social network connections or protein structures, the data in our anomaly detection scenario does not inherently form a graph. Thus, an action on our end is necessary to manually construct a graph structure using our collected network data, for GNNs to be utilized in our case.

To create a graph structure from our data, we use a kNN classification algorithm. Here, instead of using it as a classifier, we treat kNN in an unsupervised manner and for each data point in our collected dataset, we find the k data points that are the most similar in features. Then, we simply treat each data point as a node and connect the k closest data points to the node via edges.

An important question is whether the process of creating an artificial graph out of non-graph data might produce additional bias or error. Although such an undesired outcome is possible, graph construction methods seem to work without introducing substantial issues in datasets with a relatively smaller number of dimensions [23], which suits our case here. Nevertheless, for much larger datasets, one may need a more efficient approach to tackle scalability challenges [24]. We choose to go through a kNN-based graph construction method due to the simplicity of implementation and the characteristics of our collected dataset. During the study, we also attempted to use another technique for constructing graphs based on temporal traffic dissection, inspired by the method proposed in [25]. However, due to implementation issues and subpar results, this approach was not adopted and was replaced by the simpler kNN mechanism.

3) *Model Details*: For the actual GNN model, aiming to detect Kubernetes-based anomalous network flows, we employ

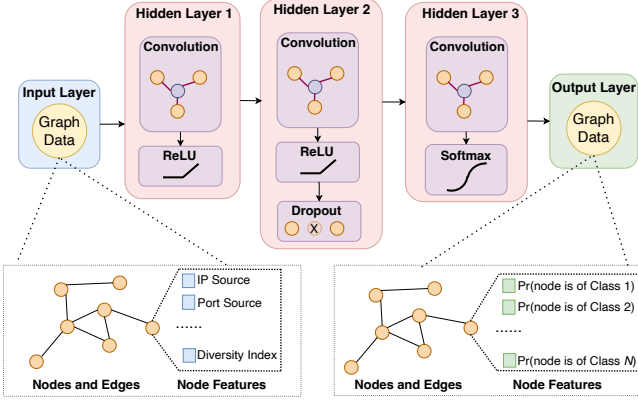


Figure 3: An overview of the GCN model in GATAKU

a GCN model with three Convolutional Layers, one Dropout Layer, ReLU as Activation Function for hidden layers and Softmax for the Output Layer. The model outputs a list of probabilities, each of which represents how likely a data point belongs to the corresponding anomaly class. The highest probability is considered when deciding the anomaly class of the target node. An overview of this model is shown in Figure 3.

IV. EXPERIMENTS

In this section, we elaborate on our experimental setup and present the results we obtain through our experiments.

A. Dataset

Using our testbed, we spin up our application in 14 different namespaces and monitor the system for about 1.5 hours. During this time period, we manually trigger three waves, two waves, and one wave of DoS, ZAP Scan, and Port Scan attacks, respectively. At the end of the monitoring phase, we collect approximately 900 MB size of OpenTelemetry data. The details of the gathered dataset and its sampled version are given in Table II.

Table II: Dataset Distribution by Anomaly Class

Anomaly Class	Number of Packets - Presampling (10^3)	Number of Packets - Postsampling (10^3)
Benign	3800	3200
DoS	900	120
Port Scan	300	50
ZAP Scan	1	1
Total	5000	3400

As one can observe, the malicious packets form approximately 25% of the entire dataset. However, this ratio is not very natural, given that anomalies are supposed to be rare events that occur much less frequently in real life. Therefore, we do a re-sampling to obtain a more realistic dataset.

After the sampling process, the proportion of malicious activity is 5% of the entire monitored network data. We conduct our experiments using this more sensible dataset.

Table III: Single-Class Experiment Results

Model	Precision		Recall		Accuracy		F1 Score	
	Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.
SVM	1.0	1e-5	0.991	0.004	0.999	2e-4	0.996	0.002
RF	1.0	1e-5	0.96	0.011	0.998	6e-4	0.979	0.006
GCN	0.999	6e-4	0.999	4e-4	0.999	1e-4	0.999	3e-4

Table IV: Multi-Class Experiment Results

Model	Precision		Recall		Accuracy		F1 Score	
	Mean	Std.	Mean	Std.	Mean	Std.	Mean	Std.
SVM	0.998	1e-4	0.998	1e-4	0.998	1e-4	0.998	1e-4
RF	0.994	2e-4	0.994	2e-4	0.994	2e-4	0.994	2e-4
GCN	0.999	1e-4	0.999	1e-4	0.999	1e-4	0.999	1e-4

B. Benchmark Methods

For the benchmarking process, we choose two traditional ML methods, RF and SVM, due to their widespread usage in various ML tasks. Our choice of benchmark models allows us to compare the performance of a GNN model against traditional ML models, in an inherently non-graph problem. Furthermore, our data is synthetically formed, and each packet is marked with a label, allowing us to leverage supervised learning.

C. Results

In this section, we display and discuss our experimental results. For each experiment, we use 90% of our dataset for the training phase and the remaining 10% for the testing phase, which is a common ratio used in literature. The split is made in a stratified manner, meaning that the proportion of anomaly classes is preserved.

We test both single-class and multi-class scenarios in our experiments. In the former case, the goal is to determine only whether a packet is anomalous or benign, while in the latter case, the models also aim to detect the specific anomaly class accurately. We present the results of each scenario separately in the following part.

1) *Single-Class*: For each of the three models, namely GCN, RF, and SVM, 10 individual trials are executed. Various metrics are recorded for each trial, per model, as can be seen in Table III. The columns “Mean” and “Std.” in the table represent the average value and the standard deviation, respectively, of the corresponding metric across trials.

We can observe that all three models perform very well in identifying the existence of an anomalous packet. In terms of precision, SVM and RF work flawlessly, achieving zero false positives. On the other hand, GCN incorrectly identifies some benign flows as malicious. Nevertheless, when other metrics such as accuracy and F1-score are considered, GCN performs slightly better than RF and SVM.

2) *Multi-Class*: For the multi-class scenario, models are trained to identify which attack class each malicious packet belongs to. Again, 10 trials are run separately for each of the three models: GCN, RF, and SVM. The recorded metrics can be seen in Table IV which follows the same column structure as Table III but with values of the multi-class simulation.

One can observe that all three models perform exceptionally well in determining malicious packets and their anomaly class. Although the difference is minimal, GCN achieves slightly better results compared to the other two models.

D. Hyperparameter Optimization

During the training phase, we ensure that our GCN model and the other benchmark models utilize an optimal configuration of hyperparameters. However, our optimization approach differs for GCN and other models.

1) *GCN Hyperparameters*: The GNN training process itself contains a validation phase, where the validation loss is backpropagated into the previous layers for adjusting specific parameters to increase performance. But, to set up the initial configuration of the model, we leverage Bayesian search to obtain the best initial hyperparameters listed in Table V. Although more exhaustive methods, such as grid search, exist for hyperparameter optimization, the complexity of neural networks usually necessitates a more efficient approach [26].

Table V: GCN Hyperparameters

Parameter	Value
Optimizer	AdamW
Learning Rate	0.03524
Weight Decay	0.00048
Epochs	29
Patience	7
Hidden Dimensions	38
Dropout	0.4416

After finalizing the initial parameters, the validation process in the training phase of GCN handles the adjustments. As mentioned before, we use 90% of our dataset for the training, and one-third of the training set is used as the validation. In other words, we use 60%, 30%, and 10% of the dataset for the training, validation, and testing phases, respectively.

2) *SVM and RF Hyperparameters*: For the benchmark methods, we optimize hyperparameters by utilizing a k-fold cross-validation approach during our experiments. For each simulation trial, we split the training dataset into three folds and use two folds for the model learning and one fold for the validation phase. Again, we use a stratified split technique to preserve the class ratios.

We use grid search to try out various combinations of hyperparameters, and for each configuration, we repeat the 3-fold cross-validation process. Then, using the best-performing configuration, we test our model on the test dataset. This entire pipeline is repeated for each of the 10 trials during our experiments. The optimal hyperparameters for SVM and RF are given in Table VI. Even though we utilize Bayesian search for the GCN optimization, we choose grid search for our baseline models. Due to the lower number of hyperparameters of RF and SVM models, grid search can be executed in a feasible amount of time. Moreover, a recent study suggests that using grid search over Bayesian search for tuning the

hyperparameters of a random forest model might provide better results [27].

Table VI: SVM and RF Hyperparameters (- : Not applicable)

Parameter	Value - SVM	Value - RF
C	10	-
Penalty	L2	-
Tol	0.0001	-
Max Depth	-	6
Number of Estimators	-	100

Note that for each trial, the set of optimal hyperparameters may change, depending on the training and validation sets. However, in our experiments, the same values turn out to be the best for at least 8 out of 10 trials, and thus, are presented.

E. Simulation Time

To give an insight into how long it takes to run the experiments, we provide our simulation times for GCN and other benchmark models. Two different devices are used for the experiments. One is a VM with Ubuntu 22.04 OS, 32 GB RAM, 16 vCPU AMD EPYC-Milan with 1 core per socket, 1 thread per core, with BogomIPS of 3992.49, and 320 GB SSD storage. The other is a mobile workstation (WS) with MacBook Pro, M3 Pro CPU, 36 GB RAM and 1 TB SSD.

Table VII: GCN Simulation Time (in seconds)

Machine	GC	DL	TpE	TT	1T	10T
Mobile WS	5400	15	12	300	5715	855
Ubuntu 22.04 (VM)	7200	40	30	750	7990	1510

The approximate simulation times of GCN are presented in Table VII where GC, DL, TpE, TT, 1T and 10T represents the time for *Graph Construction*, *Data Loading*, *Training per Epoch*, *Total Training*, *Estimated Time for 1 Trial* and *Estimated Average Trial Time for 10 Trials*, respectively. Our simulations show that a trial of the experiment takes 25 epochs on average due to the early stopping mechanism, which allows us to simply multiply the per-epoch time by 25 to estimate the total training time.

Another remark is that we only need to run the graph construction process once, and then, the constructed graph can be loaded for as many experiments as possible. Therefore, while calculating the total experiment time per trial, the time of graph construction is divided by the number of trials, and thus, its weight gradually decreases as the number of trials increases, allowing the estimated time for 1 trial to converge into a summation of total training time and data loading time.

Table VIII: Simulation Time - Benchmark Models (in seconds)

Machine	SVM Training Time	RF Training Time
Mobile WS	1350	3500
Ubuntu 22.04 (VM)	1800	4700

The simulation times for SVM and RF are provided in Table VIII. The reported results also include the time for the

hyperparameter selection process based on the 3-fold cross-validation phase.

V. DISCUSSION

In this section, we present our main findings and analyze the explainability of GATAKU to some extent.

A. Findings

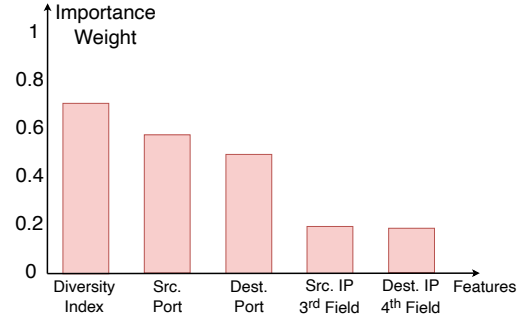
Upon examining the results of our experiments, we come up with the following findings:

- 1) *Effectiveness of GNN on network anomaly detection:* Based on the classification performance of our GCN model, GNNs can be used in detecting anomalous network flows. However, scalability and higher overhead are important challenges.
- 2) *Gain of GNN over traditional methods:* Although our simulation results indicate that our GCN model slightly outperforms some traditional ML methods such as random forest and SVM, the gain in prediction accuracy is quite minimal, raising another question on how a GNN model would perform on another dataset.
- 3) *Performance gain and training time trade-off:* There exists a visible training time difference between a GNN model and traditional ML methods. Even when we consider the hyperparameter optimization time for the traditional methods, it still takes more time to train a simple GCN model. Moreover, for training a GNN model, a graph structure from the raw input needs to be formed beforehand, introducing another computational overhead in the pipeline. Therefore, GNN is not applicable for resource-sensitive microservices environments and may be more suitable for some niche applications.

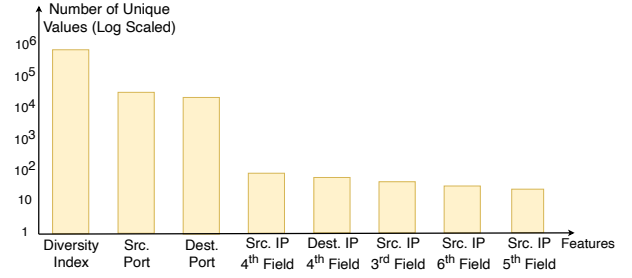
B. GNN Explainability

To gain further insights into the predictions of our GCN model, we investigate suitable AI explainability methods. Among possible candidates, we adopt GNNExplainer [28], which is one of the oldest and most widely used explainability techniques for GNN models. GNNExplainer follows a post-hoc, black-box approach where the parameters of the underlying GNN model are not known by the explainer. Instead, when the GNN model, a sample graph data, and a node to be investigated in that graph are provided as input, GNNExplainer tries to detect a subgraph and a subset of node features that are available in the test data, which contributes most to the prediction result of the target node. As output, the explainability model assigns weights, ranging from zero to one, to nodes, edges, and features, as an indication of which nodes, edges, and features are prominent for the decision made by the underlying GNN model.

Using the PyTorch implementation of GNNExplainer, we analyze a sample of our data and try to understand the basis on which our model classifies network packets. A sample of prediction is provided into the explainer, where Node-89 is predicted as a port scan attack. As output, Node-61467 is marked as important while the edge between Node-89 and Node-61467 is highlighted by GNNExplainer. In



(a) Top 5 features by GNNExplainer for Node 89



(b) Top 8 unique features in the dataset

Figure 4: Analysis of the features regarding explainability

addition, node features are ranked in terms of importance in this prediction. In Figure 4a, the top five most important features identified by GNNExplainer are shown, where the most important feature corresponds to the *diversity_index* we have constructed while other features represent IP address and port information of the source and destination of a flow. To validate the output of GNNExplainer in our case, we also conduct a data analysis on the input and plot the eight most diverse features in our dataset by the number of unique values they contain, as one can observe in Figure 4b.

The comparison shows us that, the two perspectives are mostly aligned with each other. For GNNExplainer, the most important features are diversity index, source port, destination port, the 3rd 8-bit field of the source IP, and the 4th 8-bit field of the destination IP, in order. Four of these features have the same rank in terms of unique values. The only difference is that the fourth most important feature determined by GNNExplainer, the 3rd 8-bit field of source IP, is the sixth most unique feature. On the other hand, the fourth unique feature, the 4th 8-bit field of source IP, is not considered among the top five major features by the explainer. However, we must also note that this discrepancy does not make the GNNExplainer results less reliable, as the uniqueness does not always imply significance. Furthermore, we utilize local explanations and test a single node, rather than making a global explanation for the entire model, meaning that results will likely differ for other tested instances.

VI. CONCLUSION

In this study, we proposed GATAKU, a GCN-based anomaly detection method for detecting cybersecurity attacks based

on inter-container network packets in a Kubernetes environment. We built a cloud-based infrastructure and synthetically constructed a dataset of reconnaissance and DoS attacks. The results demonstrate the GATAKU's capability to detect network anomalies with high accuracy and reliability. The performance metrics and the detailed analyses conducted validate the effectiveness of the model architecture, the hyperparameter tuning strategy, and the preprocessing methodologies employed. Moreover, we conducted an explainability analysis to provide human-interpretable explanations for the decisions made by our model.

These findings affirm the model's practical utility in network security in niche applications. We realize that the solution currently does not serve well on a bigger scale due to its high resource and time consumption compared to traditional models. However, it lays a solid foundation for future enhancements, especially regarding graph scaling and performance optimization. Furthermore, its integration to cloud systems could provide deeper insights into network traffic and potential threats, facilitating proactive defense mechanisms. Future research plans include setting up a larger testbed to investigate our scheme, testing different attack scenarios, adapting the model for online inference to perform near real-time predictions, and applying federated learning for handling privacy-preserving concerns.

ACKNOWLEDGMENT

This work has received co-funding from the Smart Networks and Services Joint Undertaking (SNS JU), the Swiss State Secretariat for Education, Research and Innovation (SERI), and the UK Research and Innovation (UKRI) under the European Union's Horizon Europe research and innovation programme, in the frame of the NETWORK project (Net-Zero self-adaptive activation of distributed self-resilient augmented services) under Grant Agreement No 101139285.

REFERENCES

- [1] Kubernetes, "Kubernetes project." Available: <https://kubernetes.io/> [Accessed: 24.08.2024].
- [2] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.
- [3] A. Gupta and R. Simon, "Enhancing security in cloud computing with anomaly detection using random forest," in *2024 11th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, pp. 1–6, 2024.
- [4] M. Tabassum, S. Mahmood, A. Bukhari, B. Alshemaimri, A. Daud, and F. Khalique, "Anomaly-based threat detection in smart health using machine learning," *BMC Medical Informatics and Decision Making*, vol. 24, p. 347, Nov 2024.
- [5] S. Naseer, Y. Saleem, S. Khalid, M. K. Bashir, J. Han, M. M. Iqbal, and K. Han, "Enhanced Network Anomaly Detection Based on Deep Neural Networks," *IEEE Access*, vol. 6, pp. 48231–48246, 2018.
- [6] B. Siegel, "Industrial Anomaly Detection: A Comparison of Unsupervised Neural Network Architectures," *IEEE Sensors Letters*, vol. 4, pp. 1–4, Aug. 2020.
- [7] H. Kim, B. S. Lee, W.-Y. Shin, and S. Lim, "Graph anomaly detection with graph neural networks: Current status and challenges," *IEEE Access*, vol. 10, pp. 111820–111829, 2022.
- [8] C. Qiu, M. Kloft, S. Mandt, and M. Rudolph, "Raising the Bar in Graph-level Anomaly Detection," *arXiv e-prints*, p. arXiv:2205.13845, May 2022.
- [9] L. Huang, Y. Zhu, Y. Gao, T. Liu, C. Chang, C. Liu, Y. Tang, and C.-D. Wang, "Hybrid-order anomaly detection on attributed networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12249–12263, 2023.
- [10] Z. You, X. Gan, L. Fu, and Z. Wang, "GATAE: Graph attention-based anomaly detection on attributed networks," in *2020 IEEE/CIC Int. Conf. on Communications in China (ICCC)*, pp. 389–394, 2020.
- [11] Y. Pei, T. Huang, W. van Ipenburg, and M. Pechenizkiy, "Resgcn: Attention-based deep residual modeling for anomaly detection on attributed networks," in *2021 IEEE 8th International Conference on Data Science and Advanced Analytics (DSAA)*, pp. 1–2, 2021.
- [12] T. Huang, Y. Pei, V. Menkovski, and M. Pechenizkiy, "Hop-count based self-supervised anomaly detection on attributed networks," *arXiv e-prints*, p. arXiv:2104.07917, Apr. 2021.
- [13] P. Kisanga, I. Woungang, I. Traore, and G. H. S. Carvalho, "Network Anomaly Detection Using a Graph Neural Network," in *2023 International Conference on Computing, Networking and Communications (ICNC)*, pp. 61–65, Feb. 2023.
- [14] C.-W. Tien, T.-Y. Huang, C.-W. Tien, T.-C. Huang, and S.-Y. Kuo, "Kubanomaly: Anomaly detection for the docker orchestration platform with neural network approaches," *Engineering Reports*, vol. 1, no. 5, p. e12080, 2019.
- [15] O. Tunde-Onadele, J. He, T. Dai, and X. Gu, "A study on container vulnerability exploit detection," in *2019 IEEE International Conference on Cloud Engineering (IC2E)*, pp. 121–127, 2019.
- [16] M. Cavalcanti, P. Inacio, and M. Freire, "Performance evaluation of container-level anomaly-based intrusion detection systems for multi-tenant applications using machine learning algorithms," in *Proceedings of the 16th International Conference on Availability, Reliability and Security, ARES '21*, (New York, NY, USA), ACM, 2021.
- [17] S. Srinivasan, A. Kumar, M. Mahajan, D. Sitaram, and GuptaSanchika, "Probabilistic real-time intrusion detection system for docker containers," in *Security in Computing and Communications* (S. M. Thampi, S. Madria, G. Wang, D. B. Rawat, and J. M. Alcaraz Calero, eds.), (Singapore), pp. 336–347, Springer Singapore, 2019.
- [18] M. Molan, M. S. Ardebili, J. A. Khan, F. Beneventi, D. Cesarini, A. Borghesi, and A. Bartolini, "Graafe: Graph anomaly anticipation framework for exascale hpc systems," *Future Generation Computer Systems*, vol. 160, pp. 644–653, 2024.
- [19] Cilium Project, "Welcome to Cilium's documentation! — Cilium 1.15.2 documentation." Available: <https://docs.cilium.io/en/stable/> [Accessed: 24.03.2024].
- [20] The Linux Foundation, "What is eBPF? An Introduction and Deep Dive into the eBPF Technology." Available: <https://ebpf.io/what-is-ebpf/> [Accessed 01.03.2024].
- [21] Cilium Project, "Hubble." Available: <https://github.com/cilium/hubble> Mar. 2024. [Accessed: 01.03.2024].
- [22] S. Cho, I. Han, H. Jeong, J. Kim, S. Koo, H. Oh, and M. Park, "Cyber kill chain based threat taxonomy and its application on cyber common operational picture," in *2018 Int. Conf. On Cyber Situational Awareness, Data Analytics And Assessment (Cyber SA)*, pp. 1–8, 2018.
- [23] S. Arya and D. M. Mount, "Approximate nearest neighbor queries in fixed dimensions," in *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '93*, (USA), p. 271–280, Society for Industrial and Applied Mathematics, 1993.
- [24] W.-L. Zhao, H. Wang, and C.-W. Ngo, "Approximate k-nn graph construction: A generic online approach," *IEEE Transactions on Multimedia*, vol. 24, pp. 1909–1921, 2022.
- [25] F. Zola, L. Seguro-Gil, J. Bruse, M. Galar, and R. Orduna-Urrutia, "Network traffic analysis through node behaviour classification: a graph-based approach with temporal dissection and data-level preprocessing," *Computers & Security*, vol. 115, p. 102632, 2022.
- [26] M. Masum, H. Shahriar, H. Haddad, M. J. H. Faruk, M. Valero, M. A. Khan, M. A. Rahman, M. I. Adnan, A. Cuzzocrea, and F. Wu, "Bayesian hyperparameter optimization for deep neural network-based network intrusion detection," in *2021 IEEE International Conference on Big Data (Big Data)*, IEEE, Dec. 2021.
- [27] R. Muzayanah, D. A. A. Pertiwi, M. Ali, and M. A. Muslim, "Comparison of gridsearchcv and bayesian hyperparameter optimization in random forest algorithm for diabetes prediction," *Journal of Soft Computing Exploration*, vol. 5, pp. 86–91, 4 2024.
- [28] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating Explanations for Graph Neural Networks," *arXiv e-prints*, p. arXiv:1903.03894, Mar. 2019.